



Examen #2

Índices y Mejoras en el Desempeño de Consultas

Bitácora de uso de IA

Camilo José Ardila Restrepo

Escuela de Ingenierías, Universidad Pontificia Bolivariana

Tópicos Avanzados en Bases de Datos

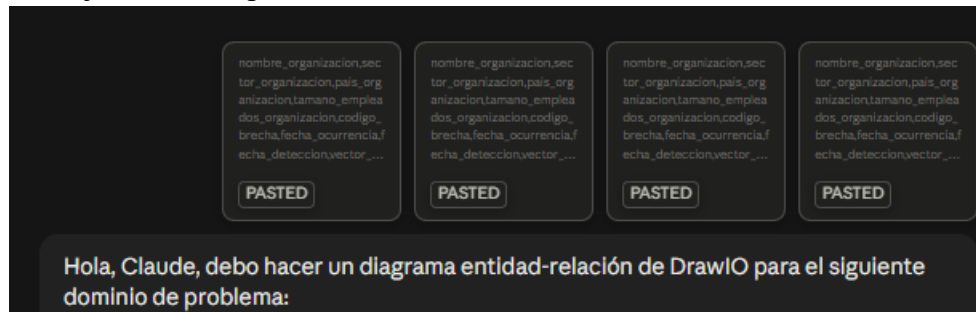
Docente: Juan Darío Rodas Marín

01 de octubre de 2026

A continuación, se detallan los principales prompts escritos por Camilo Ardila y los outputs de los agentes de IA usados para la resolución del enunciado de este examen:

1. Interacción con Claude para diagrama relacional

Se le pidió a Claude que proponga un diagrama Entidad-Relación normalizado que cumpla con los objetivos de la práctica:



(Seguida de la formulación del dominio del problema)

Asimismo, se le especifica el objetivo de la práctica y se le pide realizar un análisis de las primeras filas de los CSV para determinar tipos de datos y decisiones de diseño.

El objetivo de esta practica es hacer un performance tuning y evaluación de utilidad de índices y su utilidad y evidencia de uso de parte del motor dentro de la ejecución de las consultas. Tóma lo anterior en cuenta al proponer el modelo.

Mira los primeros campos de los CSV de exportación, úsalos para ratificar tipos de dato y campos, dime si necesitas algo más, pregúntame todo lo que tengas en duda o te falte para proponerme un gran modelo. Usa nombramiento estándar, en singular, separado por "_" y con alta especificidad en cada uno de los campos ej: Id_organizacion, nombre_organizacion

Mira la data adjunta y dame una propuesta acá en el chat y te lo apruebo:

Claude brinda sus consideraciones iniciales y decisiones de diseño

Propuesta de modelo relacional: brechas de seguridad

1. Hallazgos en los datos

- **Dependencias funcionales confirmadas:**
 - `codigo_brecha` determina organización, vector, severidad, fechas, registros y costo. Se repite idéntico entre lotes, por ejemplo BR-00740 y BR-00402.
 - `tipo_dato_expuesto` determina `categoria_sensibilidad_dato` en todas las filas de muestra. Ejemplos: Tarjeta de Crédito e Historial Médico son Crítica; Contraseña, Número de Identificación e Historial de Transacciones son Alta; Correo y Dirección Física son Media; Nombre, Teléfono y Fecha de Nacimiento son Baja.
 - `codigo_usuario` determina pseudónimo, hash, país de residencia y fecha de registro.

2. Modelo propuesto (3FN o superior)

Uso claves sustitutas enteras para los joins, más claves naturales (`codigo_*`) con índice UNIQUE. Así las tablas grandes quedan angostas y el acceso por `codigo_usuario` de la Etapa 4 usa un índice único.

Catálogos

Tabla	Columnas
<code>pais</code>	<code>id_pais</code> PK (SMALLINT), <code>nombre_pais</code> UQ
<code>sector_economico</code>	<code>id_sector_economico</code> PK, <code>nombre_sector_economico</code> UQ
<code>vector_ataque</code>	<code>id_vector_ataque</code> PK, <code>nombre_vector_ataque</code> UQ
<code>severidad_incidente</code>	<code>id_severidad_incidente</code> PK, <code>nombre_severidad_incidente</code> UQ, <code>nivel_severidad_incidente</code> UQ (1 a 4, para ordenar)
<code>categoria_sensibilidad</code>	<code>id_categoria_sensibilidad</code> PK, <code>nombre_categoria_sensibilidad</code> UQ, <code>nivel_categoria_sensibilidad</code> UQ (1 a 4)

(Seguido de una especificación más detallada de todo el diagrama propuesto y las deudas o vacíos de conocimiento que tiene)

Luego de leer el output de claude y analizar el diagrama, se halla una marca “FK?”, que claude hacía para especificar un vacío en el contexto de su información, por lo que se le cuestiona acerca de ella

¿Qué significa "FK?" en usuario? Asimismo, tienes toda la cardinalidad y multiplicidad considerada en el diagrama?

Thought for 9s

"FK?" en `usuario`

Es una marca mía, no notación estándar. Indica que `id_organizacion` en `usuario` es una llave foránea todavía sin confirmar. El enunciado dice que cada usuario está asociado a una organización, pero en las filas que vi un mismo usuario aparece con varias organizaciones (por ejemplo, US-005333 y US-006654 traen distinta organización). Depende de la consulta de la pregunta 4 sobre la sábana completa:

Viendo que Claude tiene algunas dudas de diseño que quiere validar, se le cuestiona acerca de cómo debería hacerse, sabiendo que lo que se tiene son archivos .csv:

¿Cómo podemos validar eso si no tengo nada importado en una base de datos?
Solo tengo los CSV

Thought for 14s

Se puede validar sin instalar ni cargar nada en un servidor de base de datos.

Corrección a mi mensaje anterior

Puse como evidencia que US-005333 y US-006654 traían organizaciones distintas. Es un error mío: son dos usuarios diferentes, no el mismo. En las ~100 filas de muestra ningún `codigo_usuario` se repite, así que **la muestra no permite concluir nada** sobre la pregunta 4. Con 40,000 filas y 7,887 usuarios sí habrá repetidos en la sábana completa, y ahí se puede comprobar.

Opciones para validar con solo los CSV

Opción 1: DuckDB (la más rápida). Es un motor SQL de un solo archivo que lee los CSV directamente. No hay servidor ni configuración.

1. Descomprime los cuatro `.zip` en una carpeta.
2. Instala DuckDB con `pip install duckdb`, o descarga el ejecutable desde duckdb.org.
3. Abre `duckdb` en esa carpeta y ejecuta:

Se decide instalar DuckDB y hacer validaciones rápidas mediante la creación de una DB temporal, por lo que se le especifican a Claude las rutas de los .csv y este responde con una guía de abastecimiento rápida de DuckDB:

Mira la ruta a mis CSV y su contenido para Importar

PS C:\Users\camil\OneDrive\Documents\2026-20\TABD\tadb202620_examen_02\data> ls

Directory: C:\Users\camil\OneDrive\Documents\2026-20\TABD\tadb202620_examen_02\data

Mode	LastWriteTime	Length	Name
-a----	9/25/2026 8:49 AM	2412032	sabana_brechas_seguridad_lote_1.csv
-a----	9/25/2026 8:49 AM	2413783	sabana_brechas_seguridad_lote_2.csv

Show more

3 days ago

Thought for 3s

Como ya estás en esa carpeta, lo más simple es usar DuckDB desde PowerShell, con rutas relativas.

1. Instalar DuckDB

Elige una de las dos formas:

```
powershell
winget install DuckDB.cli
# o, si tienes Python:
pip install duckdb
```

Con `winget`, cierra y vuelve a abrir PowerShell para que reconozca el comando `duckdb`.

2. Abrir DuckDB en la carpeta `data`

Estás parado en `...\tadb202620_examen_02\data`. Ejecuta:

```
powershell
duckdb sabana.duckdb
```

Esto crea `sabana.duckdb` en esa carpeta y no modifica tus CSV. Si prefieres no guardar nada, ejecuta solo `duckdb`.

3. Cargar y validar

Pega este bloque en el prompt `D`:

```
sql
-- Carga: todo como texto, para no perder acentos ni fechas raras
```

Luego de instalar DuckDB, se le muestran a Claude los mensajes que arrojó la terminal, tras esto, el modelo responde con una serie de consultas de utilidad para validar nuestras decisiones de diseño:

```

enter ".help" for usage hints.
sabana D .mode markdown
sabana D
sabana D CREATE OR REPLACE TABLE stg_sabana AS
        SELECT * FROM read_csv('sabana_brechas_seguridad_lote_*.csv',
        header = true, all_varchar = true, quote = '"', filename = true);
sabana D
sabana D SELECT COUNT(*) AS filas,
        COUNT(DISTINCT nombre_organizacion) AS organizaciones,
        COUNT(DISTINCT codigo_brecha) AS brechas.
Show more

```

Los conteos coinciden con el enunciado: 40,000 filas, 60 organizaciones, 800 brechas, 7,887 usuarios y 10 tipos de dato. La carga y el manejo de comillas funcionaron.

Para no pegar diez consultas por separado, este bloque devuelve una sola tabla con el número de violaciones de cada regla. Pégallo en el prompt:

```

sql

SELECT 'P3a: fecha_notificacion no depende solo de (brecha, usuario)' AS chequeo, COUNT(*)
  FROM stg_sabana GROUP BY codigo_brecha, codigo_usuario
  HAVING COUNT(DISTINCT fecha_notificacion_usuario) > 1)
UNION ALL
SELECT 'P3b: duplicados (brecha, usuario, tipo_dato)', COUNT(*) FROM (
  SELECT 1 FROM stg_sabana GROUP BY codigo_brecha, codigo_usuario, tipo_dato_expuesto
  HAVING COUNT(*) > 1)
UNION ALL
SELECT 'P4: usuarios con mas de una organizacion', COUNT(*) FROM (
  SELECT 1 FROM stg_sabana GROUP BY codigo_usuario

```

Se le comparten los resultados de las consultas, pidiéndole sus conclusiones acerca del diseño según lo que estas muestran:

```
PS C:\Users\camil\OneDrive\Documents\2024-2025\TAD\Tad2\2024-2025\examen_02\data> & "C:\Users\camil\OneDrive\Documents\duckdb_cli_windows-...
```

PASTED

Mira, dime cuales son tus conclusiones finales

Thought for 13s

Conclusiones

Los resultados cambian una decisión importante de mi propuesta y confirman el resto.

1. Mi recomendación de la opción A era incorrecta.

La consulta P3a dio **5,026 violaciones**: para un mismo par (brecha, usuario), la `fecha_notificacion_usuario` cambia según el tipo de dato expuesto. La fecha depende de la llave completa (brecha, usuario, tipo de dato), no solo de la pareja. Por eso la opción B es la correcta y además coincide con las 5 tablas que pide el enunciado. La P3b dio 0, así que (brecha, usuario, tipo de dato) es una llave válida.

2. Modelo final: 10 tablas.

Desaparecen las dos tablas puente separadas y queda una sola:

Tabla	Cambio
<code>brecha_usuario_tipo_dato</code>	PK compuesta (<code>id_brecha_seguridad</code> , <code>id_usuario</code> , <code>id_tipo_dato_expuesto</code>) y <code>fecha_notificacion_usuario</code> <code>DATE NOT NULL</code> . Reemplaza a <code>afectacion_usuario_brecha</code> y <code>afectacion_tipo_dato</code> .
<code>usuario</code>	<code>id_organizacion</code> se queda como FK obligatoria y quito el signo <code>?</code> . P4 dio 0: ningún usuario aparece con más de una organización.
Las 8 tablas restantes	Sin cambios.

Luego de abordar las decisiones y consideraciones de diseño relevantes, se le pide a Claude un diagrama relacional que las tenga en cuenta:

0

sabana D

Dame la revisión del diagrama

Ran a command, shared files

Actualicé `modelo_brechas_seguridad.drawio`: ahora tiene 10 tablas y 11 relaciones. Validé la estructura del XML, pero no lo pude abrir en draw.io desde aquí.

Sobre tu último resultado

Mi consulta estaba mal diseñada. Agrupaba por (usuario, brecha) y contaba organizaciones, y eso siempre da 0 porque cada brecha tiene una sola organización, así que no probaba nada. Lo que sí se puede concluir viene de dos resultados anteriores: cada usuario tiene una sola organización ($P4 = 0$) y cada brecha también (la dependencia de brecha dio 0). Como la sábana trae una sola columna de organización por fila, cada usuario solo aparece en brechas de su propia organización. Por eso `usuario.id_organizacion` se queda como FK obligatoria, sin signo de interrogación.

Cambios en el diagrama

- Tabla puente única: `brecha_usuario_tipo_dato`, con PK compuesta de (`id_brecha_seguridad`, `id_usuario`, `id_tipo_dato_expuesto`) y `fecha_notificacion_usuario`. Reemplaza a las dos tablas anteriores, porque la validación mostró que la fecha depende del tipo de dato.
- Multiplicidades con mínimos: el lado padre es siempre "exactamente uno". El lado hijo es 0..N en casi todas las relaciones. Desde `brecha_seguridad` y `usuario` hacia la tabla puente es 1..N, y desde `tipo_dato_expuesto` es 0..N.
- Tipos ajustados a los datos: `VARCHAR(10)` para los códigos, `VARCHAR(50)` para el pseudónimo y los nombres de catálogo, y `VARCHAR(100)` para el nombre de organización.
- Recuadro de restricciones: incluye los `CHECK`, la regla de notificación que se valida en la carga, la ausencia de `CHECK` sobre la fecha de registro y la regla de negocio no forzada por la BD.
- Leyenda: ahora explica la notación de multiplicidad.

Decisiones que te pido revisar

- 1..N hacia la tabla puente: la base de datos no lo puede forzar. Lo dejé como regla de negocio documentada. Si prefieres que el diagrama sea estrictamente lo que impone la BD, lo cambio a 0..N.
- Redundancia de `usuario.id_organizacion`: podría derivarse por la tabla puente. La dejé porque el enunciado asocia cada usuario a una organización, y además acelera consultas por organización.
- Definiciones de la Etapa 3: sigo con los supuestos por defecto (fecha de detección, `CURRENT_DATE` como referencia, todos los usuarios distintos de la brecha). Un detalle a considerar: la sábana llega hasta 2026-09-22 y hoy es 2026-09-30, así que "último año" con `CURRENT_DATE` cubre prácticamente todo el rango reciente. Dime si prefieres otra referencia.

Si el diagrama te convence, el siguiente paso es el DDL con índices, vistas y `CHECK`, el script de carga desde `stg_sabana` (que puedo escribir para DuckDB y PostgreSQL) y las consultas de las Etapas 3 y 4 con `EXPLAIN`. ¿Con cuál motor quieres trabajar la entrega?

Modelo brechas seguridad

DRAWIO

Download

Claude responde con un diagrama relacional de DrawIO, sin embargo, también pide revisar ciertos aspectos del mismo. Luego de analizar lo anterior, se concluye que no hay nada bloqueante, por lo que simplemente se le pide a la herramienta una lista explicativa para agregar como cuadro de texto en el diagramador:

No me parece mal las decisiones que me pediste revisar, dame una lista pequeña en la que se expliquen, agregaré un cuadro de texto especificandolas para que queden aclaradas

Mi compañero revisa esta versión del diagrama y me pide preguntarle al modelo acerca de un detalle que la base de datos permitiría en el estado actual:

Ah, mira esta consideración que me acaban de dar unos compañeros:

usuario tiene `id_organizacion` y `brecha_seguridad` también. En la tabla puente, un usuario llega a su organización por dos caminos: el propio y el de la brecha. Con los datos actuales coinciden en el 100% de las filas (0 diferencias), pero el modelo no lo garantiza. Nada impide insertar en la puente un usuario de la organización A afectado por una brecha de la organización B.

¿Está abordada o requiere ajustes?

Thought for 10s

No estaba abordada. Vi la redundancia y la dejé como decisión 3 ("se conserva por conveniencia"), pero no cerré el hueco de integridad que describen tus compañeros. Tienen razón en que los 0 casos de diferencia son una propiedad de los datos y no del modelo. Hoy la base aceptaría insertar un usuario de la organización A afectado por una brecha de la B, y la regla "el usuario solo aparece en brechas de su organización" quedaría sin garantía.

Claude recomienda usar una FK compuesta para la tabla "brecha_usuario_tipo_dato", lo que aborda la consideración de buena manera. Con esto, se le pide actualizar el diagrama para reflejar la decisión:

Prosigue con tu recomendación, si vas a hacer un nuevo diagrama, deja el cuadro de abajo a la izquierda con el siguiente texto en lugar del que tiene.

También, haz que el recuadro de "Restricciones adicionales" pase a ser el resumen de las decisiones de diseño más importantes.

Convención de colores y notación:

Azul: catálogos, Verde: entidades, Naranja: tabla puente

PK: llave primaria, FK: llave foránea (NOT NULL), UQ: única

Lado padre: exactamente uno. Lado hijo: cero o muchos (0..N), o uno o muchos (1..N) hacia la tabla puente desde brecha y usuario

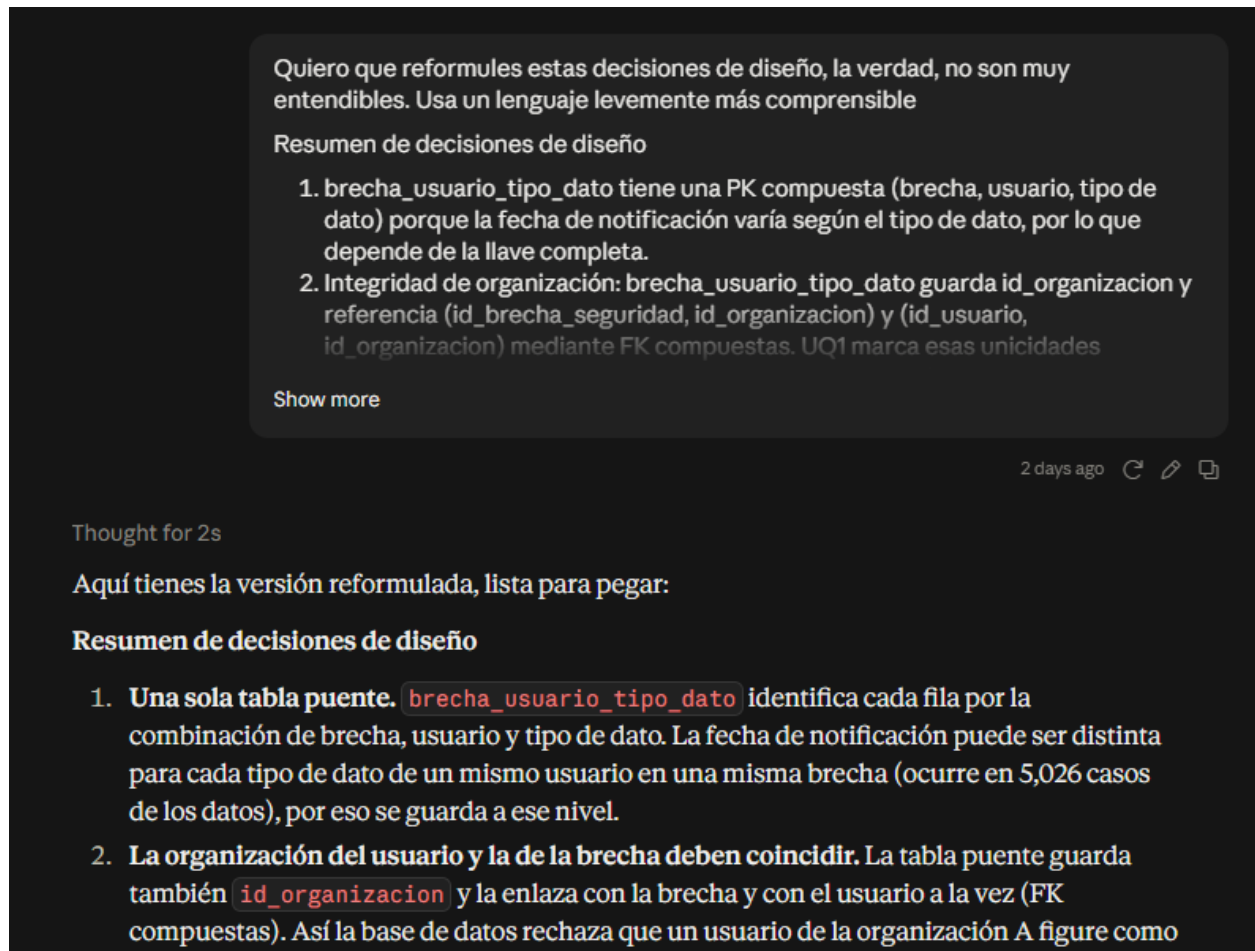
2 days ago

Ran 2 commands, shared files

Actualicé `modelo_brechas_seguridad.drawio` con la opción A. Validé la estructura del XML, pero no lo pude abrir en draw.io desde aquí.

Finalmente, se le pide reformular las decisiones de diseño para mayor explicabilidad en la

documentación:



2. Interacción con Claude para el montaje de la base de datos en nube:

Se le detalla a Claude la necesidad de desplegar una DB en AWS con conexiones seguras:

Hola, Claude, quiero crear una base de datos de AWS para correr una MySQL. Necesito crear una instancia de Aurora con un security group que permita conexiones desde mi máquina únicamente (Voy a estar moviéndome de red, entre la de mi casa, universidad, etc, consideralo). Asimismo, necesito conexiones cifradas en caso de ser necesarias y parámetros de seguridad simples (No necesito nada respecto al esquema SQL y su seguridad por el momento, solo la seguridad de la nube, supongo que todo se puede manejar desde un security group con reglas simétricas)

Comparando el script dinámico frente al túnel SSM. >

Antes de lanzarme a los comandos, hay un punto que vale la pena aclarar porque cambia la arquitectura: un security group de AWS filtra por **IP de origen**, no por "mi máquina". Si te mueves entre casa, universidad, cafés, etc., tu IP pública cambia constantemente, así que una regla de SG con tu IP fija se rompe cada vez que cambias de red. No es que el security group no pueda ser simple — es que "solo mi máquina" y "solo mi IP" dejan de ser lo mismo en cuanto tu IP cambia.

(En el prompt se le dijo MySQL, pero luego se decidió cambiarla a PostgreSQL)

Claude responde con las opciones de acceso y conexión a la misma, preguntando por cuál me decantaré.

¿Qué arquitectura de acceso prefieres?

Sería posible usar una llave SSH? No quisiera usar una bastión, tampoco tengo tanto problema con exponerla, puedo hacer más manejos desde el connectionString

¿Cómo prefieres los comandos?

Consola web paso a paso

Verificando el soporte actual de RDS para certificados de cliente. >

Aclaro un punto importante antes de armar los pasos: **RDS/Aurora no tiene acceso SSH** — no hay sistema operativo expuesto, es un servicio totalmente administrado. El "SSH key" tal como lo usas para EC2 no aplica aquí. Lo que sí existe como equivalente real (un par de llaves/certificado que el cliente debe presentar para autenticarse) es **autenticación por certificado de cliente X.509 (mutual TLS)** en MySQL. Déjame confirmar el estado actual de soporte de eso en Aurora MySQL antes de darte pasos exactos, porque es un detalle que cambia entre versiones de motor.

También, el modelo brinda una guía de creación rápida para un cluster en AWS:

1. Crear el cluster Aurora MySQL

1. Consola → RDS → Databases → Create database
2. Choose a database creation method: *Standard create*
3. Engine options: *Amazon Aurora* → *Aurora MySQL-Compatible Edition* (elige la versión LTS más reciente disponible)
4. Templates: *Dev/Test* si es para desarrollo (más barato) o *Production* si necesitas Multi-AZ
5. Settings: DB cluster identifier, usuario admin, y **contraseña fuerte** (o deja que RDS la gestione con Secrets Manager, casilla "Manage master credentials in AWS Secrets Manager" — recomendado, rota sola)
6. Instance configuration: elige la clase (`db.t4g.medium` es un buen punto de partida económico)
7. Connectivity:
 - Compute resource: "Don't connect to an EC2 compute resource"
 - Network type: IPv4 (o dual-stack si lo necesitas)
 - Virtual private cloud (VPC): la que uses
 - Public access: Yes (esto es lo que la hace alcanzable desde internet)
 - VPC security group: *Create new* → nómbralo, por ejemplo `aurora-mysql-sg`
 - Availability Zone: déjalo automático
8. Additional configuration (parte de abajo del todo):
 - Encryption: marca "Enable encryption" y deja la KMS key por defecto (`aws/rds`) — esto cifra el almacenamiento en reposo. **Importante: esto solo se puede activar en la creación, no después.**
 - Backup: deja el retention period por defecto (7 días) o ajústalo
9. Create database

Luego, se le especifica el cambio a PgSQL y se le pide repetir la guía:

Ah, espera, me acaban de dar luz verde para usar PostgreSQL, repíteme los pasos para hacer este proceso con los requerimientos que te he dicho

Luego de intentar aplicar la guía, se llega al siguiente error en la consola debido a que se estaba intentando instanciar desde una cuenta educacional de laboratorio de AWS:

Ah, mira esto, creo que la cuenta no permite rds:

User: arn:aws:sts::530325636486:assumed-role/voclabs/user3795408=camiloardila is not authorized to perform: rds:CreateDBInstance on resource: arn:aws:rds:us-east-1:530325636486:db:database-tadb2 because no identity-based policy allows the rds:CreateDBInstance action

Valídalo y dime qué deberíamos hacer

yesterday ↻ ✎ 📄

Narrowing the lab's scope to Aurora-only database creation. >

Confirmado, y esto encaja con la hipótesis: el lab está **específicamente scoped a Aurora**, no a RDS "clásico". El patrón que viste es el típico de un Learner Lab — la política IAM del rol `voclabs` permite las acciones de Aurora (`rds:CreateDBCluster`, `rds:CreateDBInstance` dentro del flujo de Aurora) pero no permite crear una instancia RDS standalone del motor PostgreSQL puro. Por eso llegaste hasta el final del wizard sin bloqueos — la consola no valida permisos IAM hasta el submit — y ahí te tiró el `AccessDenied`.

(Claude sugiere usar el Free Tier de AWS en lugar de una cuenta educacional)

Luego, se accede a AWS desde una cuenta gratuita y se le muestra lo que esta permite hacer:

Configuration	Value 	Modifiable post-creation
DB engine version	Version 17	Yes, upgradable
DB cluster identifier	database-1	Yes
Database master username	postgres	Yes
DB instance type	Serverless* Automated vertical (up/down) scaling	Yes
Min capacity value	0 ACU (0 GiB) (Scales to 0 after 30min of inactivity)	Yes
Max capacity value	4 ACU (8 GiB) 1 to 4 in increments of 0.5	Yes
Pause after inactivity	300 seconds 300 to 16,800 seconds (5 minutes to 24 hours)	Yes
Storage configuration	Aurora Standard*	Yes
Encryption	Enabled with AWS/RDS owned key	No
Internet access gateway	Enabled	No
Private access/VPC	Disabled/No VPC used	No
Authentication	IAM only	No

Entré a una cuenta free tier y mira lo que me deja crear:

yesterday   

Reevaluando cómo la autenticación IAM resuelve el requisito de seguridad. >

Esto confirma que llegaste al **"Create with express configuration"** de Aurora Free Tier que habíamos visto en la documentación — pero fíjate que la arquitectura de seguridad es **radicalmente distinta** a todo lo que armamos hasta ahora. Vale la pena que lo revisemos antes de darle a "Create", porque cambia varias cosas.

Claude afirma que esta es la alternativa que debemos usar, eligiendo entre una full creation o express creation del cluster. Así, se le especifica al modelo el uso de un full creation, consecuentemente, Claude responde con una nueva guía de creación:

Usé full create, eligiendo PostgreSQL nativo

Reviewing the RDS PostgreSQL setup details for the request.

Perfecto, eso simplifica las cosas — es el mismo flujo de RDS PostgreSQL "clásico" que armamos antes de la pelea con Aurora. Los pasos pendientes son los de seguridad que quedaron colgados:

1. Security Group — abrir el puerto 5432

1. EC2 → Security Groups → busca el SG que se creó junto con tu instancia (aparece en la pestaña **Connectivity & security** de tu base de datos en RDS, campo "VPC security group")
2. Inbound rules → Edit inbound rules → Add rule:
 - Type: PostgreSQL (auto-rellena puerto 5432)
 - Source: Anywhere-IPv4 (0.0.0.0/0)
3. Save rules

2. Forzar TLS (conexiones cifradas)

Como es RDS estándar (no Aurora), el parameter group es a nivel de instancia:

1. RDS → Parameter groups → Create parameter group
 - Parameter group family: postgres17 (o la versión que elegiste)
 - Type: DB Parameter Group
 - Nombre: pg-force-ssl
2. Ábrelo → Edit parameters → busca rds.force_ssl → cámbialo a 1 → Save changes
3. RDS → Databases → selecciona tu instancia → Modify
 - DB parameter group → selecciona pg-force-ssl
 - Apply immediately → Continue → Modify DB instance
4. Actions → Reboot la instancia para que el parámetro tome efecto

3. Verificar que el cifrado en reposo quedó activo

Ve a tu instancia en RDS → pestaña **Configuration** → busca **Encryption** → debe decir "Enabled". Si por algún motivo no lo marcaste al crearla, no se puede activar después — tendrías que recrearla marcando "Enable encryption" en el paso de creación.

4. Conectarte con TLS

Descarga el certificado CA: en el menú lateral de RDS → Certificates → descarga global-bundle.pem (o el específico de tu región).

(La guía de creación se sigue al pie de la letra y esta es exitosa)

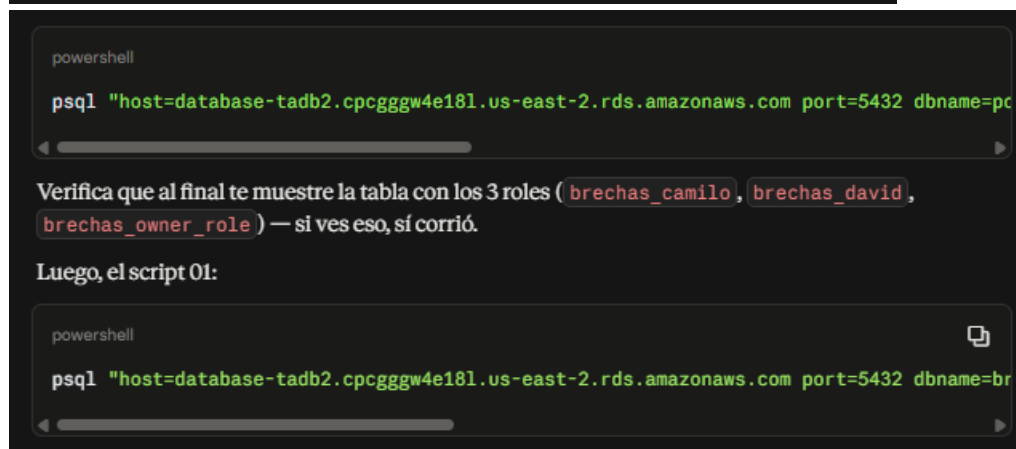
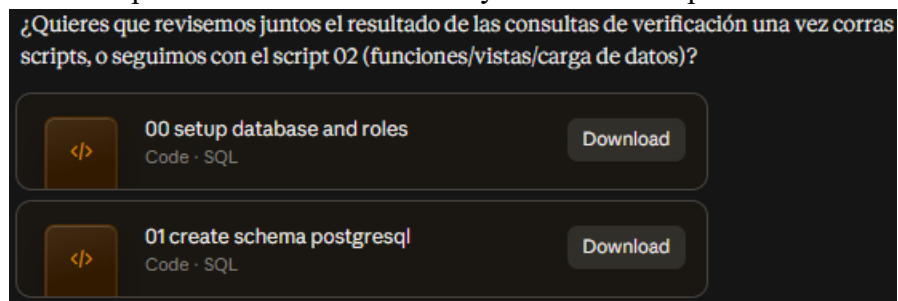
3. Interacción con Claude para adaptar scripts y ejecuciones de MySQL a PostgreSQL:

Primeramente, se le especifica a Claude la necesidad de crear una base de datos en la instancia de nube del paso anterior y abastecerla con un esquema. Por ello, se le pide crear un script para la base de datos y parsear el de mi compañero para que sea compatible con PostgreSQL:



(Uso del mismo chat que en el paso anterior)

Claude responde con ambos archivos y los comandos para subirlos:



Luego, se intentan cargar los scripts a la base de datos, sin embargo, parecen haber ciertos errores en los comandos y archivos generados, así que se le comparte en output de la terminal al modelo para que lo diagnostique:

```

PS C:\Users\camil\OneDrive\Documents\2026-20\TABD\tadb202620_examen_02\PostgreSQL-AWS\Scripts>
PS C:\Users\camil\OneDrive\Documents\2026-20\TABD\tadb202620_examen_02\PostgreSQL-AWS\Scripts> psql "host=database-tadb2.cpcgggw4e18l.us-east-2.rds.amazonaws.com port=5432 dbname=postgres user=postgres sslmode=verify-full sslrootcert=./global-bundle.pem" \
| -f 00_setup_database_and_roles.sql

Password for user postgres:
psql (18.2 server 18.3)
Show more

```

Tracing how shell line-continuation broke the script execution. >

Dos problemas distintos, ninguno grave:

1. El script 00 nunca se ejecutó. En PowerShell el continuador de línea es el **backtick** (```), no la barra invertida (`\`) como en bash/Linux. Por eso, aunque viste el prompt `>>` (PowerShell pensó que la línea seguía incompleta), el `-f archivo.sql` no se interpretó como argumento de `psql` — simplemente te conectó interactivo (por eso viste `postgres=>` y tuviste que salir con `\q`). El script de creación de la base y roles **no corrió**, por eso ahora `brechas_seguridad` no existe.

Claude modifica levemente los comandos que había brindado inicialmente y ahora el script 00 sólo ejecuta, sin embargo parece haber un error de permisos en el script 01, así que se le pide corregirlo:

```

PS C:\Users\camil\OneDrive\Documents\2026-20\TABD\tadb202620_examen_02\PostgreSQL-AWS\Scripts> psql "host=database-tadb2.cpcgggw4e18l.us-east-2.rds.amazonaws.com port=5432 dbname=postgres user=postgres sslmode=verify-full sslrootcert=./global-bundle.pem" -f 00_setup_database_and_roles.sql
Password for user postgres:
pg_terminate_backend
-----
(0 rows)

Show more

```

23 hours ago

Drafting the schema-ownership grant needed to fix permissions. >

Buen progreso — el script 00 sí corrió bien esta vez (los 3 roles quedaron creados). El error en el 01 es un detalle de seguridad que cambió en **PostgreSQL 15+**: ya no se le otorga el privilegio `CREATE` sobre el esquema `public` a todo el mundo por defecto. Antes, cualquier rol podía crear objetos en `public`; desde PG15, solo el dueño del esquema (en este caso `postgres`, porque fue quien corrió `CREATE DATABASE`) puede hacerlo, a menos que se le otorgue explícitamente a otro rol.

(Claude modifica un par de líneas en el script 01 para corregir el error)

Luego, se le comparte la salida de la terminal para ratificar la creación y subida del esquema:

```
PS C:\Users\camil\OneDrive\Documents\2026-20\TABD\tadb202620_examen_02\PostgreSQL-AWS\Scripts> psql "host=database-
tadb2.cpcgggw4e18l.us-east-2.rds.amazonaws.com port=5432
dbname=brechas_seguridad user=postgres sslmode=verify-full sslrootcert=./global-
bundle.pem" -f 01_create_schema_postgresql.sql
Password for user postgres:
GRANT
SET
CREATE TABLE
CREATE TABLE
Show more
```

22 hours ago

Setting up database tables and verifying foreign key relationships.

Excelente — el esquema quedó completo y correcto. Los 10 ejes de la tabla salen bien, y las 10 tablas están con `brechas_owner_role` como dueño, que era el objetivo clave para que `brechas_camilo` y `brechas_david` puedan crear índices.

Claude responde con el script solicitado:

02 load data postgresql
Code · SQL

Download

(El script se ejecuta manualmente y este funciona correctamente, subiendo todos los registros)

Ahora, se le pide a Claude parsear los procedimientos almacenados y funciones de la base de datos, pasando del script de MySQL a PostgreSQL:

03_funciones...
as_mysql.sql
290 lines

SQL

Perfecto, ahora, el siguiente scripts, las funciones y procedimientos propios de la db. Quiero crearlos desde camilo_brechas, comprueba que tengamos permisos y reescribe este para que funcione para nuestro caso.

Claude menciona ciertas consideraciones, pero devuelve el script parseado y el comando para ejecutarlo:

1. Permisos: si los tienes, sin nada adicional. `brechas_camilo` es miembro de `brechas_owner_role`, y ese rol ya tiene `CREATE` sobre el esquema `public` (otorgado en el script 00) y es dueño de las tablas. Crear una función solo exige `CREATE` sobre el esquema + `USAGE` sobre el lenguaje (plpgsql ya está disponible para todos por defecto) — así que no hace falta ningún `GRANT` nuevo. Eso sí, igual que con las tablas, voy a envolver la creación en `SET ROLE brechas_owner_role` para que las funciones/vistas queden con ese rol como dueño (y no con `brechas_camilo` directamente), así David también tiene control total sobre ellas sin necesitar grants extra.

2. Los "procedimientos" de MySQL no se traducen a `CREATE PROCEDURE` de PostgreSQL. Aunque PostgreSQL sí tiene procedimientos (`CREATE PROCEDURE` + `CALL`, desde la versión 11), están pensados para lógica transaccional (hacer `COMMIT` / `ROLLBACK` dentro), no para devolver un result set como hace tu `sp_brechas_criticas_ultimo_anio`. Un `CALL` en PostgreSQL no te deja hacer `SELECT * FROM resultado_del_call`. La forma idiomática de lograr "una consulta parametrizada reutilizable que devuelve filas" en PostgreSQL es una función que retorna `TABLE(...)`, invocada con `SELECT * FROM nombre(parametros)` en vez de `CALL`. Mantengo los nombres originales (`sp_...`) para que sepas que vienen de ahí, pero son funciones.

Un tercer detalle menor: tu `sp_historial_exposicion_usuario` en MySQL devuelve a veces una fila de "mensaje" y a veces la tabla de detalle — en PostgreSQL una función tiene una sola forma de salida fija, así que cuando el usuario no existe, la función lanza un `RAISE NOTICE` (verás el aviso en tu cliente) y devuelve 0 filas, en vez de una fila con texto.

Created a file, shared files >

Para correrlo, conectado como `brechas_camilo`:

powershell

```
psql "host=database-tadb2.cpcgggw4e181.us-east-2.rds.amazonaws.com port=5432 dbname=br
```

Si quieres confirmar el permiso antes de correr el script completo, puedes verificarlo con una sola línea:

powershell

```
psql "host=database-tadb2.cpcgggw4e181.us-east-2.rds.amazonaws.com port=5432 dbname=br
```

Debería devolver `t` (true).



03 funciones vistas postgresql
Code · SQL

Download



Luego, se le pide a Claude crear el script #02, que incluya una tabla temporal para subir la información de los archivos .csv y poblar nuestras tablas mediante consultas a la misma.

Perfecto, todo funcionó. Ahora es tiempo del script #2.

Todo esto lo ejecuté en C:\Users\camil\OneDrive\Documents\2026-20\TABD\tadb202620_examen_02\PostgreSQL-AWS\Scripts, dónde tengo mis scripts. Ahora, en C:\Users\camil\OneDrive\Documents\2026-20\TABD\tadb202620_examen_02\data, tengo los 4 csv con la información de abastecimiento, junto a un archivo de guardado de duckdb que debemos ignorar.

Con esto, haciendo uso de sus rutas relativas (rutas de sabana_brechas_seguridad_lote_1.csv (lo mismo para los otros 3, sumándole 1 al último dígito), quiero que hagas un script para cargar toda la data a la base de datos desde el usuario brechas_camilo. Para esto, usa una tabla temporal que tenga la misma información que el csv. Te recuerdo los campos

nombre_organizacion,sector_organizacion,pais_organizacion,tamano_empleados_organizacion,codigo_brecha,fecha_ocurrencia,fecha_deteccion,vector_ataque,severidad_incidente,registros_afectados_total,costo_estimado_total,codigo_usuario,pseudonimo_usuario,email_hash_usuario,pais_residencia_usuario,fecha_registro_usuario,tipo_dato_expuesto,categoria_sensibilidad_dato,fecha_notificacion_usuario.

Controla errores por si los 4 csv no tienen los mismos campos, con esta gran tabla, haz la población de los datos y luego de que estos queden cargados y normalizados en sus respectivas tablas, bórrala.

Show less

22 hours ago

Weighing efficiency tradeoffs in reading header rows via copy. >

Para esta carga voy a usar el meta-comando `\copy` de psql (no `COPY` del servidor) porque `brechas_camilo` no es superusuario y los CSV están en tu máquina, no en el servidor — `\copy` lee el archivo desde el cliente y lo transmite, así que funciona con privilegios normales.

Para el control de errores de "columnas distintas", valido el encabezado real de cada archivo (como texto crudo, comparado contra el esperado) antes de intentar parsearlo como CSV — así, si un archivo tiene columnas en otro orden, de más, de menos, o mal escritas, el script se detiene con un mensaje claro en vez de cargar datos desalineados silenciosamente.

Una suposición que hice y que debes confirmar: para `severidad_incidente` y `categoria_sensibilidad_dato`, tu tabla original exige un nivel numérico (1-4) que el CSV plano no trae — solo el nombre. Asumí la escala Baja=1, Media=2, Alta=3, Crítica=4 (coherente con los comentarios de tu script 01). Si tus CSV usan otras etiquetas, el script fallará explícitamente diciéndote cuáles valores no reconoció, para que los ajustes.

Se le comparte el output del comando del script 03 a claude y se le pide parsear el último script, el de las consultas:

04_consultas...
as_mysql.sql

SQL

F:\C:\Users\camil\OneDrive\Documents\2026-20\TABD\tadb202620_examen_02\PostgreSQL-AWS\Scripts> psql "host=databases-tadb2.cpcggw4e1bl..."

PASTED

Revisa que sea correcto, finalmente, este es el script de las consultas, cámbalo a pg

Claude aclara que debe hacer un workaround debido a las características del motor, pero devuelve el archivo y su comando de ejecución

Confirmado: PostgreSQL no tiene índices invisibles, ni siquiera en Aurora/RDS — la documentación oficial de AWS lo dice explícitamente, y el propio equipo de PostgreSQL discutió agregar esta función en 2024 pero no se ha implementado (el hilo de la comunidad recomienda literalmente lo mismo que voy a usar: **borrar el índice y volverlo a crear** para medir el efecto, en vez de intentar "ocultarlo"). El truco de bajo nivel que a veces se menciona (tocar `pg_index.indisvalid` directamente) está desaconsejado por el propio proyecto y de todas formas requiere privilegios de superusuario reales, que RDS no entrega ni siquiera al usuario `postgres`.

Esto cambia la metodología: en vez de `INVISIBLE` / `VISIBLE`, la "línea base" se obtiene con `DROP INDEX`, y la "mejora" recreando el índice exacto del script 01.

Otros cambios de fondo en la traducción:

Created a file, shared files >

Para correrlo:

```
powershell  
psql "host=database-tadb2.cpcggw4e181.us-east-2.rds.amazonaws.com port=5432 dbname=br
```

Como es un script repetible (`DROP INDEX IF EXISTS` / `CREATE INDEX` al final de cada etapa), puedes correrlo varias veces sin problema — siempre termina con los tres índices restaurados exactamente como quedaron en el script 01.

04 consultas brechas postgresql
Code · SQL

Download

Se le pide al modelo una buena manera para medir el impacto de los índices en las consultas del anterior script, especificándole el requerimiento de las mediciones:

Perfecto, ¿Cómo debería medir el efecto?

Me conectaré desde DBeaver o tal vez podemos exportar los planes de ejecución individuales y analizarlos de mejor manera, ¿Qué recomendas?

Ah, mira el requerimiento, creo que en ese caso, lo mejor es usar DBeaver

- Capturas de pantalla donde se debe observar CLARAMENTE el árbol de operaciones generado, las filas afectadas y el valor del costo.
- Registros resultantes luego de la ejecución de la consulta

Dime cómo debería ejecutar todo y dónde puedo encontrarlo en DBeaver

Claude responde con una guía para consultar y medir el impacto de los índices haciendo uso del script 04:

Con ese requisito específico ("árbol de operaciones, filas, costo" + registros resultantes), DBeaver te da exactamente eso con su panel nativo de plan de ejecución — no necesitas el JSON/pev2 que te sugerí antes, eso era más para un análisis fino de I/O. Aquí va el flujo completo mapeado a lo que ya tienes en `04_consultas_brechas_postgresql.sql`.

Dónde está en DBeaver

- Atajo: `Ctrl+Shift+E` con el cursor dentro de la consulta (sin escribir `EXPLAIN` tú mismo — DBeaver lo agrega internamente).
- O en el menú: botón "Explain Execution Plan" en la barra de herramientas del editor SQL.
- El resultado abre una pestaña nueva con el árbol de operaciones; cada fila tiene columnas de Cost y Rows, y al hacer clic en una fila del árbol, el panel de la derecha muestra el detalle completo de ese nodo — eso es justo lo que te piden mostrar en la captura.

Flujo paso a paso por etapa

Para cada etapa (3 y 4), vas a repetir este patrón dos veces (línea base y mejora):

1. Ejecuta el `DROP INDEX` correspondiente del script (selecciona solo esa línea, `Ctrl+Enter`).
2. Ejecuta `ANALYZE` de las tablas (siguiente línea del script).
3. Pon el cursor sobre la consulta `SELECT` grande (la del árbol de JOINS) y presiona `Ctrl+Shift+E` → se abre el árbol → **captura de pantalla aquí** (línea base, estimado).
4. Para el componente de "filas reales" (no solo estimadas), ejecuta directamente como consulta normal (`Ctrl+Enter`, no el atajo de explain):

```
sql
EXPLAIN (ANALYZE, BUFFERS) SELECT ...;
```

Esto te da una sola fila con el plan en texto, incluyendo tiempos y filas reales — amplía la celda o ábrela en el "Value Viewer" (panel inferior/lateral de DBeaver al seleccionar la celda) para que se vea completo en la captura.

5. Ejecuta el `CREATE INDEX` (bloque de "mejora", incluyendo el `SET ROLE / RESET ROLE`).
6. Repite los pasos 3 y 4 sobre la misma consulta → esta vez el árbol debería mostrar `Index Scan / Bitmap Index Scan` en vez de `Seq Scan`, y el costo debería bajar — **captura de pantalla aquí** (mejora).
7. Finalmente, ejecuta la consulta sin `EXPLAIN` (la sección "3.3"/"4.3" del script) con `Ctrl+Enter` normal → esa es tu captura de "registros resultantes".

Para la Etapa 4 con la variable de psql

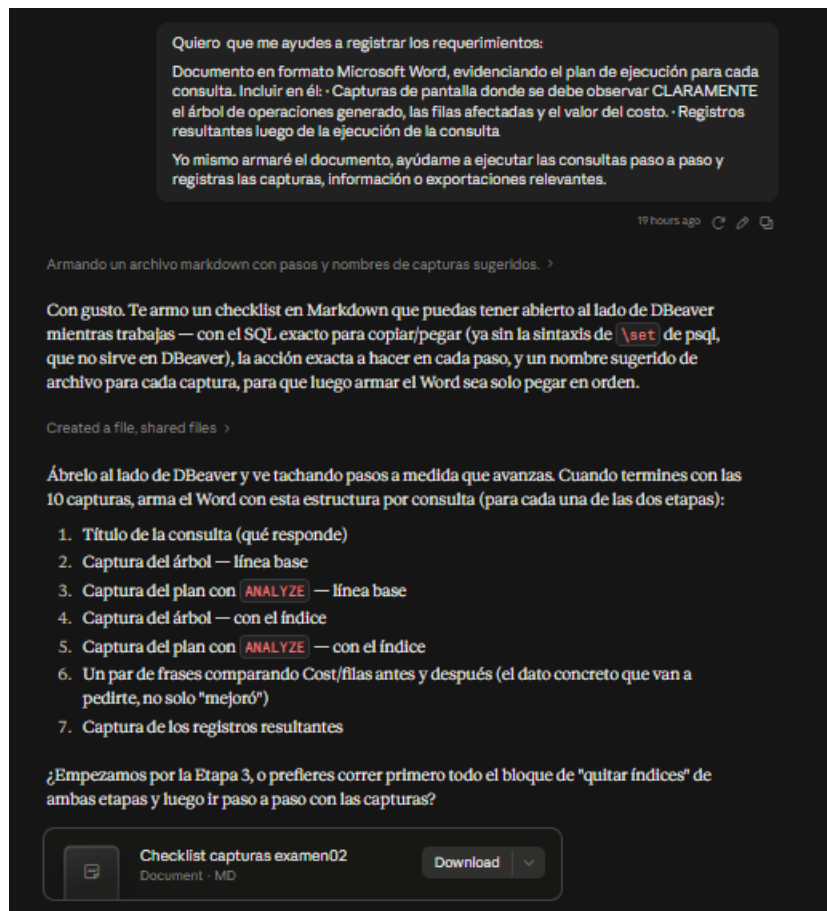
Ojo: la sintaxis `\set codigo_usuario 'US-002823'` y `:'codigo_usuario'` es específica de psql, no de DBeaver — DBeaver no entiende esos metacomandos. En DBeaver simplemente reemplaza `:'codigo_usuario'` por el literal directo en cada consulta:

```
sql
WHERE u.codigo_usuario = 'US-002823'
```

Y lo mismo en la llamada final a la función:

```
sql
SELECT * FROM sp_historial_exposicion_usuario('US-002823');
```

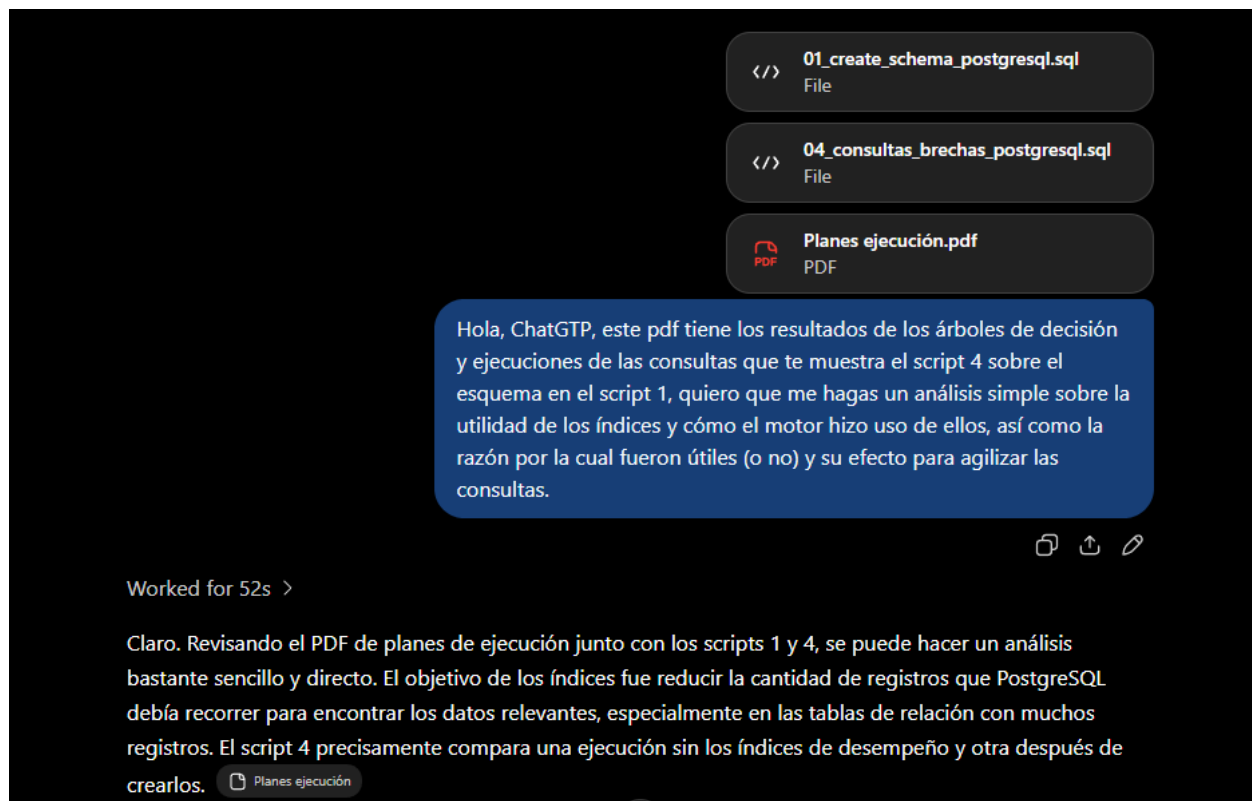
Finalmente, le solicitamos a Claude plasmar la anterior guía en formato markdown para poder ejecutar las consultas y tomar buenas evidencias del impacto de los índices.



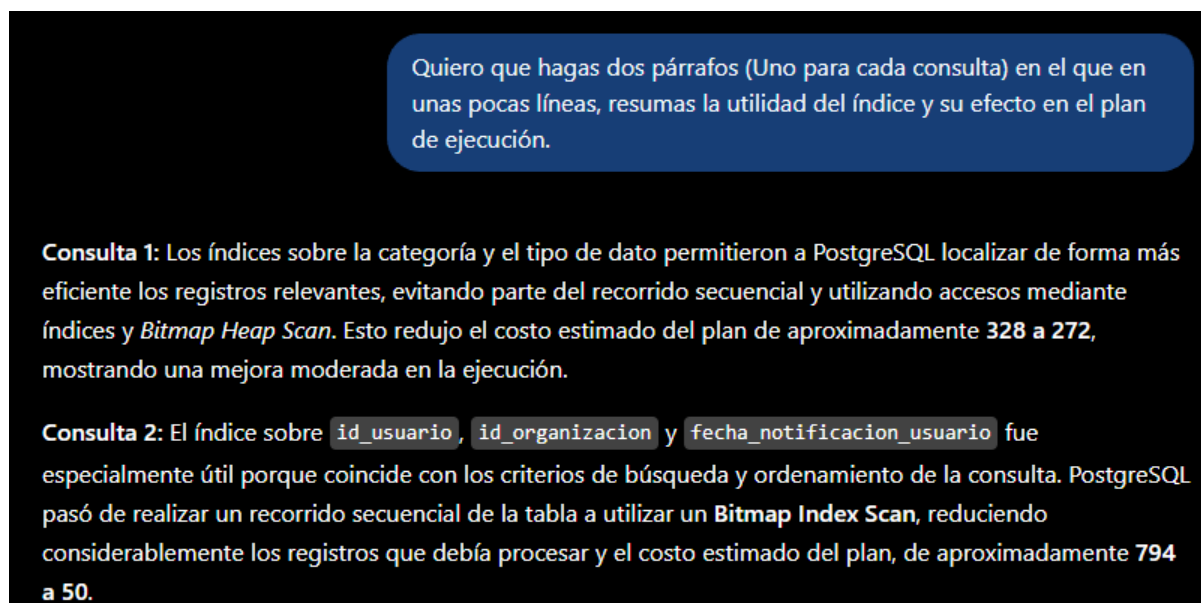
(Se sigue la guía del markdown, realizando la documentación solicitada)

4. Interacción con ChatGPT para analizar y documentar los planes de ejecución:

Se le pidió a ChatGPT que analizara los planes de ejecución y las consultas realizadas para determinar la eficiencia y evidencia de la utilidad de los índices implementados en las mismas.



Luego de un análisis rápido, se le pidió resumirlo en 2 párrafos pequeños para incluirlos brevemente en la documentación.



Finalmente, se le pidió parsear los CSV de los outputs de las consultas a tablas para pegarlas en

la documentación:



Resultado_Etapa3_Brechas_Criticas_Ca...
Spreadsheet



Resultado_Etapa4_Historial_US-00282...
Spreadsheet

Perfecto, ahora, estos son los resultados de las consultas, quiero que me los des en tablas listas para pegar en un informe de word

Claro. Te dejo las dos consultas en formato de tabla, conservando los resultados de los CSV y con encabezados adecuados para un informe de Word.